

PATRÓN DE DISEÑO: INTERPRETER

Sistema de Gestión — Centro Educativo "Educar para Transformar"

Equipo de Trabajo: NovaSys — Bruno Mazza, Fabián Trangoni

Nombre del Patrón: Interpreter (Intérprete)

Tipo de Patrón: Comportamiento

Descripción del Problema

En el sistema de gestión del centro educativo existen reglas de negocio que combinan varias condiciones y que suelen cambiar o ampliarse con el tiempo. Por ejemplo, para inscribir a un alumno a un deporte es necesario validar en conjunto que no supere el máximo de dos deportes y que no existan choques de horario; para asignar un profesor a un curso hay que verificar disponibilidad y que la materia no tenga ya un docente asignado. Si estas reglas se codifican mediante estructuras condicionales (if/else) directamente en el código, cada nueva combinación de condiciones obliga a modificar y volver a probar la lógica existente, lo que vuelve al sistema rígido y difícil de mantener. El problema que resuelve Interpreter es representar estas reglas como una pequeña gramática interpretable, de modo que puedan combinarse y ampliarse sin alterar el código ya probado.

Descripción de la Solución

El patrón propone representar cada regla del sistema como una clase que implementa un método `interpret()`. Las reglas simples (expresiones terminales) evalúan una condición puntual contra los datos del alumno o de la inscripción; las reglas compuestas (expresiones no terminales, como AND, OR o NOT) combinan el resultado de otras reglas, terminales o compuestas. De esta manera, una regla de negocio se arma combinando piezas reutilizables, en lugar de escribirse como un bloque de código nuevo cada vez.

Participantes

- **Expresión Abstracta:** interfaz que declara el método `interpret()`, común a todas las reglas.
- **Expresiones Terminales:** reglas básicas del sistema, por ejemplo `ReglaMaximoDeportes`, `ReglaSinChoqueHorario` o `ReglaNivelIgual`.
- **Expresiones No Terminales:** combinan otras expresiones mediante operadores lógicos, por ejemplo `ExpresionAND`, `ExpresionOR` o `ExpresionNOT`.
- **Contexto:** contiene los datos concretos sobre los que se evalúa la regla (el alumno, sus inscripciones actuales, el deporte u horario a validar).
- **Cliente:** arma el árbol de expresiones necesario para una regla de negocio puntual y ejecuta `interpret()` sobre el Contexto.

Colaboración

El Cliente construye el árbol combinando expresiones terminales y no terminales según la regla que necesita validar. Cada expresión no terminal invoca recursivamente el `interpret()` de sus expresiones hijas y combina sus resultados con la operación lógica correspondiente. Las expresiones terminales consultan directamente el Contexto para obtener el dato puntual que necesitan y devuelven el resultado de su condición. La evaluación se propaga desde la raíz del árbol hasta las hojas, y el resultado final surge de combinar las respuestas parciales.

Consecuencias

- Permite agregar nuevas reglas de negocio sin modificar las reglas ya existentes (principio abierto/cerrado).
- Favorece la reutilización: una misma regla básica (por ejemplo, "sin choque de horario") puede combinarse en distintas validaciones del sistema.
- Facilita la lectura de las reglas complejas, ya que se arman combinando piezas simples con un significado claro.
- Si el número de reglas crece mucho, la cantidad de clases puede volverse difícil de administrar; para gramáticas más complejas conviene evaluar herramientas de parsing en lugar de aplicar el patrón manualmente.

Ejemplo de Aplicación

Al inscribir un alumno a un deporte, el sistema debe validar la regla compuesta "máximo 2 deportes" Y "sin choque de horario". Usando Interpreter, esta regla se arma como:

ExpresionAND(ReglaMaximoDeportes(alumno, limite=2), ReglaSinChoqueHorario(alumno, nuevoDeporte))

Al ejecutar `interpret(contexto)`, la expresión AND evalúa ambas subreglas contra los datos del alumno y combina el resultado. Si más adelante se agrega una nueva condición (por ejemplo, tener la ficha médica al día), alcanza con crear una nueva expresión terminal e incorporarla a la combinación con AND, sin modificar las reglas ya existentes.

Bibliografía

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Refactoring.Guru. (s.f.). Interpreter. Recuperado de <https://refactoring.guru/es/design-patterns/interpreter>

SourceMaking. (s.f.). Interpreter Design Pattern. Recuperado de https://sourcemaking.com/design_patterns/interpreter

INTERPRETER

Reglas de negocio que se leen como una gramática — aplicado al sistema de gestión de "Educar para Transformar".

NovaSys · Bruno Mazza & Fabián Trangoni

// 01 — el problema

Una regla nueva, un *if* más

Inscribir a un alumno en un deporte exige validar en conjunto dos condiciones: que no supere el máximo de dos actividades y que no haya choque de horario. Escritas a mano, cada nueva condición obliga a tocar código ya probado — y el sistema se vuelve rígido y difícil de mantener.

// 02 — la solución

Cada regla, una pieza de lenguaje

Interpreter convierte cada condición en una clase con un método `interpret()`. Las reglas simples (terminales) leen un dato puntual del alumno; las reglas compuestas (AND, OR, NOT) combinan el resultado de otras reglas. Una validación se arma combinando piezas reutilizables, en vez de escribirse de cero.



// 03 — quién participa

Participantes

● Expresión Abstracta	Interfaz común a todas las reglas; declara el método <code>interpret()</code> .
● Expresiones terminales	Reglas atómicas del sistema: <code>ReglaMaximoDeportes</code> , <code>ReglaSinChoqueHorario</code> , <code>ReglaNivelIgual</code> .
● Expresiones no terminales	Combinan otras reglas mediante operadores lógicos: AND, OR, NOT.
● Contexto	Los datos concretos sobre los que se evalúa la regla: el alumno y sus inscripciones actuales.
● Cliente	Arma el árbol de expresiones necesario y ejecuta <code>interpret()</code> sobre el Contexto.

// 04 — consecuencias

Lo que se gana, lo que hay que cuidar

A favor

- + Nuevas reglas de negocio sin tocar las existentes.
- + Piezas básicas reutilizables entre distintas validaciones.
- + Cada regla se lee con un significado claro.

A tener en cuenta

- Muchas reglas distintas implican muchas clases.
- Gramáticas grandes piden un parser dedicado, no el patrón a mano.

Bibliografía — Gamma, Helm, Johnson & Vlissides (1994), *Design Patterns*, Addison-Wesley · refactoring.guru/es/design-patterns/interpreter · sourcecmaking.com/design_patterns/interpreter